TITLE

Project Description:

- In a single paragraph explain about the project.

Scenario:

System Requirements:

To ensure smooth development, deployment, and usage of the <—Project Name —> Web Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable application.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System**: Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Node.js (v16 or above)**: Provides the runtime environment for building and managing frontend logic and Powers the server-side logic and handles API routing. 👉 Download Node.js
- **npm (v8 or above)**: A package manager required to install dependencies for React and related libraries.
- **React.js**: A JavaScript library for building dynamic and responsive user interfaces.
- **Browser**: Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.

- **Express.js**: A lightweight web framework for building RESTful APIs.
- **MongoDB**: A NoSQL database used to store structured and unstructured data related to <— name of the collections —> 👉 Download MongoDB
- **Postman**: Tool for testing APIs during development.
- **Visual Studio Code**: Preferred code editor with built-in Git and terminal support.
- **Git & GitHub**: For version control and collaborative development.

2. Hardware Requirements

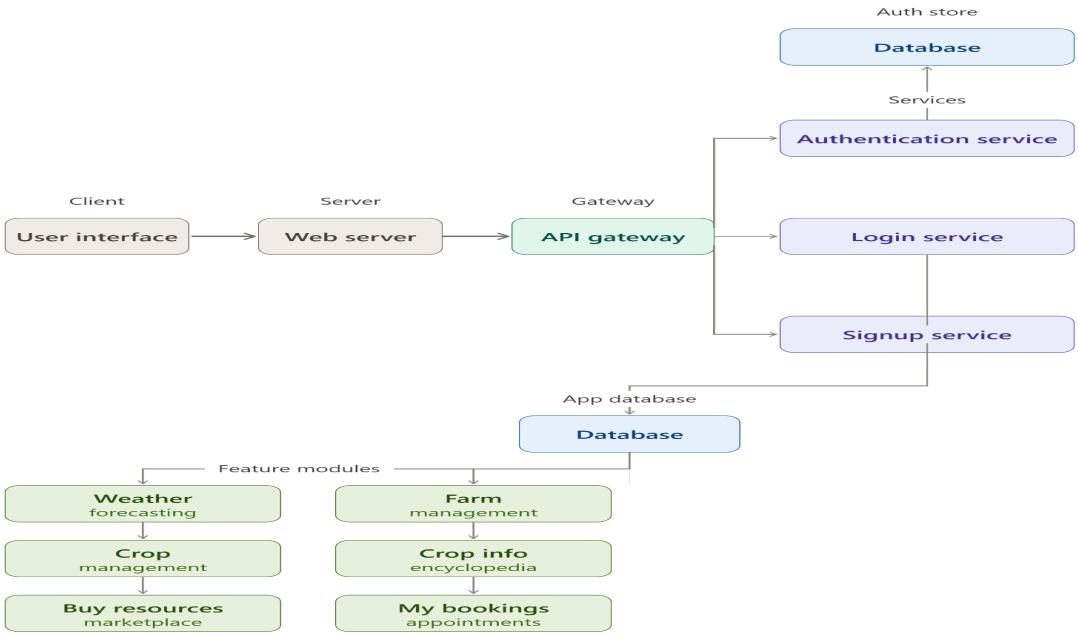
Describes the minimum and recommended specifications needed to support the development and usage of the application.

- **Processor**: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
- **RAM**: Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- **Storage**: At least 1 GB free space — Required for package installations, MongoDB setup, and local project files.
- **Display**: 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.

Epic 1 :- Project Architecture: 1hr

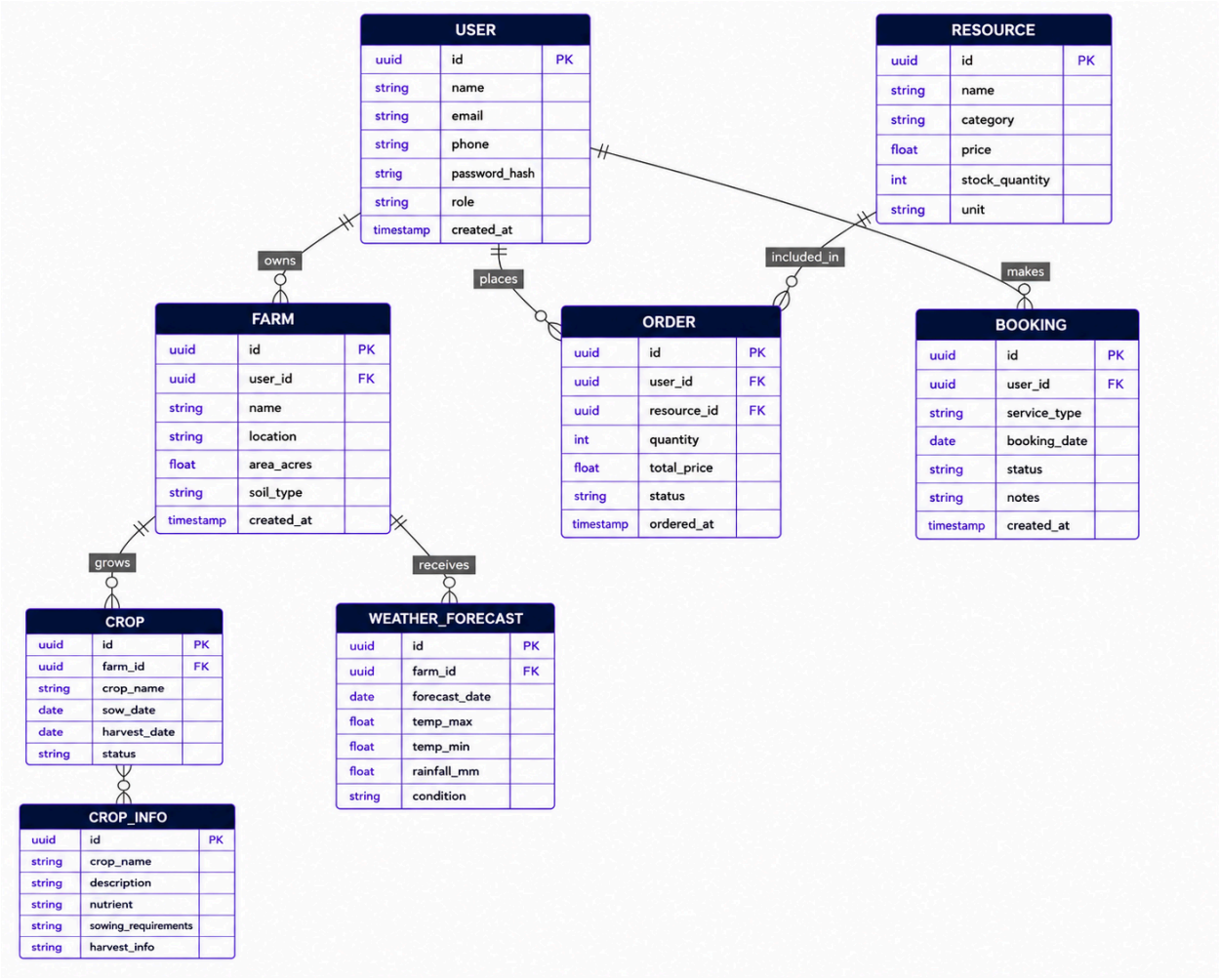
Project architecture defines the structured design of components, their interactions, and technologies used to build and deliver the application. Let's dive deep into this project's architecture

- **Technical Architecture: (Sample)**

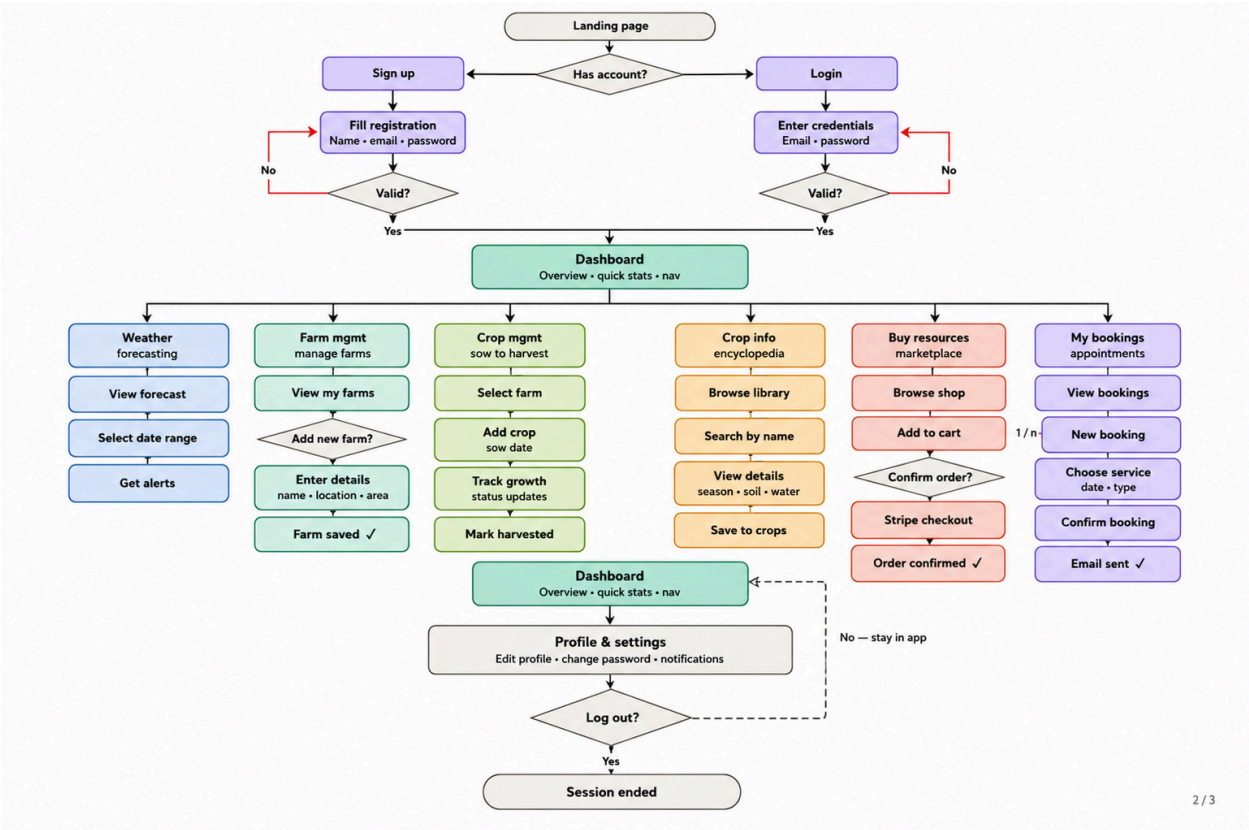


- In-detail explanation

- **ER Diagram:(Sample)**

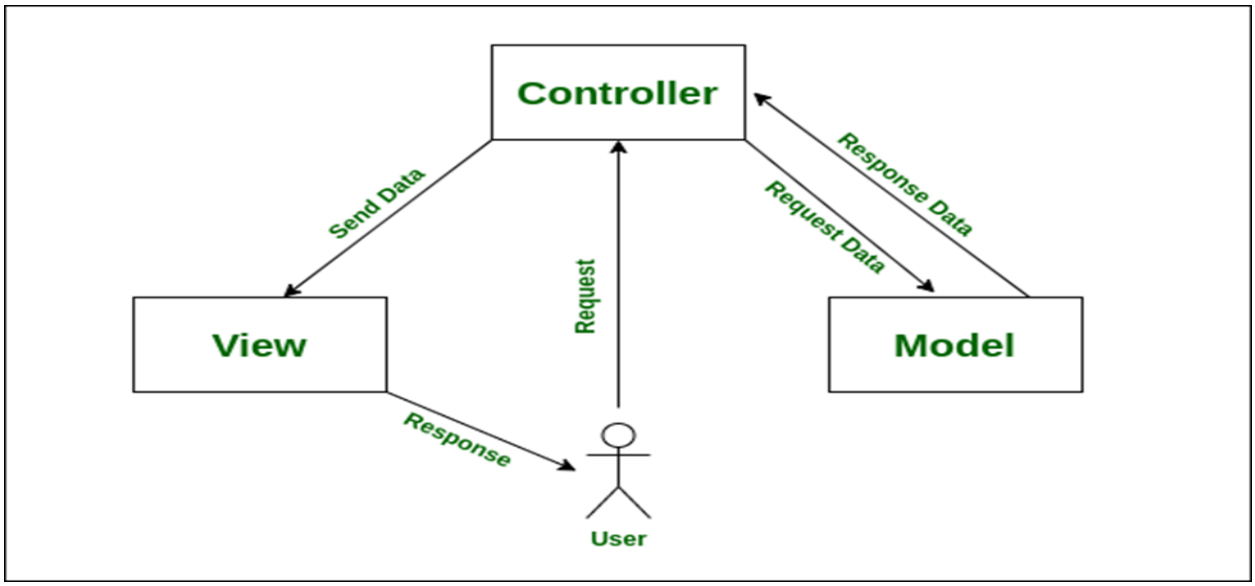


    - In-detail explanation
- **User Flow(Sample)**



- In-detail explanation

- **Features**
- **Roles and responsibilities**
- **MVC Pattern**



MVC Architecture Pattern

The backend application is designed using the Model-View-Controller (MVC) architectural pattern, a widely adopted software design approach that promotes separation of concerns by organizing the application into three distinct layers: Model, Controller, and View (Routing Layer). This architecture improves code organization, maintainability, scalability, and collaboration among development teams.

Model Layer (Data Layer)

The Model Layer is responsible for managing the application's data and business entities. It defines the structure, relationships, validation rules, and constraints for the data stored in the database.

In this project, the Model layer is implemented using Mongoose, an Object Data Modeling (ODM) library for MongoDB. Mongoose provides schema-based data modeling, enabling developers to define data structures and perform database operations efficiently.

The responsibilities of the Model layer include:

- Defining database schemas and data models.
- Managing relationships between collections.
- Performing CRUD (Create, Read, Update, Delete) operations.
- Applying data validation and business rules.
- Interacting directly with the MongoDB database.

By centralizing data-related logic within the Model layer, the application ensures data consistency and simplifies database management.

Controller Layer (Business Logic Layer)

The Controller Layer acts as the bridge between incoming client requests and the application's data models. It contains the core business logic of the application and coordinates the flow of data between the routing layer and the database.

When a request is received, the controller:

1. Accepts and processes the request data.
2. Validates and transforms input where necessary.
3. Invokes appropriate Model methods to perform database operations.
4. Applies business rules and application logic.
5. Formats and sends the appropriate response back to the client.

This separation ensures that business logic remains independent of routing and database implementation details, resulting in cleaner and more maintainable code.

View Layer (Routing Layer)

In a backend REST API application, there is typically no traditional View layer responsible for rendering user interfaces. Instead, the Routing Layer serves a similar role by defining API endpoints, receiving HTTP requests, and directing them to the appropriate controller functions. While it is not a true View in the classical MVC sense, it acts as the entry point for client interactions with the backend application.

The routing layer is responsible for:

- Defining RESTful API endpoints.
- Mapping HTTP methods (GET, POST, PUT, PATCH, DELETE) to controller functions.
- Managing middleware execution such as authentication, authorization, validation, and logging.
- Forwarding requests to the appropriate controllers.

This layer serves as the entry point for all client interactions with the backend system and ensures proper request handling and endpoint organization.

Request Flow in MVC Architecture

The typical request lifecycle within the application follows these steps:

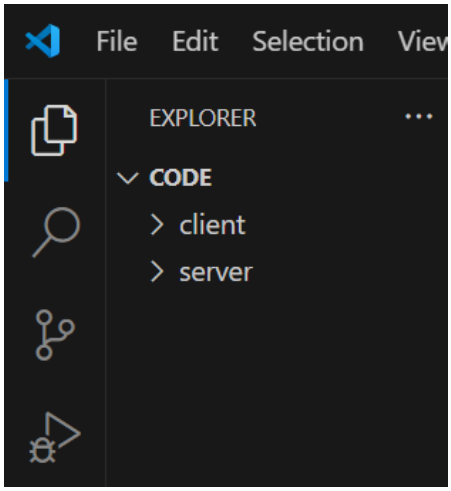
1. The client sends an HTTP request to a specific API endpoint.
2. The Route receives the request and forwards it to the corresponding Controller.
3. The Controller processes the request and invokes the necessary Model operations.

4. The Model interacts with the MongoDB database and returns the requested data.
5. The Controller formats the result and sends a response back through the Route.
6. The client receives the final response.

This structured workflow ensures clear responsibilities and efficient communication between application layers.

Epic 2:- Project Setup And Configuration. 30mins

- ### Creating project folder
    1. Create a new folder with your <project name>.
    2. Inside that folder create two new folders.
    3. Name one as **Client**.
    4. Name another one as **Server**.
    5. Now open that folder in VS Code.



- ### Client setup (installing react app)
    - Open the Client folder in the terminal of VScode.
        - npm create vite@latest . -- --template react
    - Select React framework from the given options.
        - Select a framework:
        - │ React

    - Select JavaScript variant from the given options.
        - Select a variant:
        - ❘ JavaScript
    - Now lets navigate to the client folder by giving the following command.
        - cd client
    - To install all the packages run the following command.
        - npm install
    - To start the React server type the following command.
        - npm run dev
- **Server setup (npm init)**
    - Open Server folder in terminal of VScode.
        - npm init -y
    - Create files:
        - server.js
    - Create folders:
        - models
        - controllers
        - routes

Epic 3 :- Backend Development: 8hrs

- Reference video with voiceover or screen shots for backend code

  Note:- Proper Explanation needs to be there for each and every third-party-API consumed.

Epic 4 :- Database Development 2hrs

- Reference video with voiceover or screen shots for Database code

Epic 5:- Front-End Development. 8hrs

- Reference video with voiceover or screen shots for frontend code

  Note:- Proper Explanation needs to be there for each and every third-party-API consumed.

Epic 6:- Project Execution. 30mins

- **Steps for Project execution**

  Step 1: Set Up the Frontend (React App):

  a) Open a terminal and navigate into the client folder:

  cd client

  b) Once installation is done, start the React development server:
  npm run dev

  c) The app should now be running on:
  http://localhost:5173

  Step 2: Set Up the Backend (Express Server)

  a) Open a new terminal tab/window or split the terminal.

  b) Navigate into the server folder:
  cd ../server

  Step 3: Configure Environment Variables

  a) **Inside the** server f**older, create a new file named .**env **(no file extension).**

  b) **In that** .env **file, add your MongoDB connection string:**
  MONGO_URI=mongodb://localhost:27017/<— dbname—>

  Step 4: Start the Backend Server:

- Inside the same server folder, run the backend server using**:**

  nodemon index.js
- The server should start on:

  http://localhost:8000

- **Output ScreenShots with explanation:**
- **Demo video link**
- **Code drive link**